

Online Store Order Processing

You just got hired as a junior developer at "ShopMaster", a growing online store. Your manager gives you a list of features to build for the store's backend system. The system needs to handle products, orders, notifications, and reports. "We need this system to be flexible. Tomorrow we might add new product categories, new discount rules, new notification channels, new report formats... so don't hardcode anything. I want to be able to plug in new behavior without changing existing code — making the system easy to extend without modification."

Starter Code: Data Models & Product Catalog

Before building any features, you need to set up the data models. Copy the following Product class and product catalog into your project. This is shared starter code — all subsequent tasks will use this data.

1. Product Model

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Category { get; set; } // "Electronics", "Clothing", "Food", "Books"
    public double Price { get; set; }
    public int Stock { get; set; }
}
```

2. Product Catalog

```
List<Product> catalog = new()
{
    new Product { Id=1, Name="Laptop", Category="Electronics", Price=1200, Stock=10 },
    new Product { Id=2, Name="Phone", Category="Electronics", Price=800, Stock=25 },
    new Product { Id=3, Name="T-Shirt", Category="Clothing", Price=30, Stock=100 },
    new Product { Id=4, Name="Jeans", Category="Clothing", Price=60, Stock=50 },
    new Product { Id=5, Name="Chocolate", Category="Food", Price=5, Stock=200 },
    new Product { Id=6, Name="Coffee Beans", Category="Food", Price=15, Stock=80 },
    new Product { Id=7, Name="C# Book", Category="Books", Price=45, Stock=30 },
    new Product { Id=8, Name="Novel", Category="Books", Price=20, Stock=60 },
    new Product { Id=9, Name="Headphones", Category="Electronics", Price=150, Stock=40 },
    new Product { Id=10, Name="Jacket", Category="Clothing", Price=120, Stock=15 }
};
```

Task 01 : Smart Product Search

Manager: "Customers search in all kinds of ways - by category, by price, by name, by stock... and the list keeps growing. I need ONE search method that works for any filter, now and in the future, without being modified".

What You Need To Do: Write a single method called `SearchProducts` that accepts two parameters:

1. The product list (`List<Product>`)
2. A delegate representing the filter condition (`Func<Product , bool>`)

The method should return a `List` containing only the products that satisfy the condition. Then, call this method four times with different lambda expressions to perform the following searches:

1. All Electronics products
2. Products cheaper than \$50
3. Products that are in stock (`Stock > 0`)
4. Clothing products under \$100

Expected Output:

```
--- Electronics ---  
Laptop - $1200 (Stock: 10)  
Phone - $800 (Stock: 25)  
Headphones - $150 (Stock: 40)  
  
--- Under $50 ---  
T-Shirt - $30 (Stock: 100)  
Chocolate - $5 (Stock: 200)  
Coffee Beans - $15 (Stock: 80)  
Novel - $20 (Stock: 60)  
  
--- In Stock ---  
Laptop - $1200 (Stock: 10)  
Phone - $800 (Stock: 25)  
T-Shirt - $30 (Stock: 100)  
Jeans - $60 (Stock: 50)  
  
--- Clothing Under $100 ---  
T-Shirt - $30 (Stock: 100)  
Jeans - $60 (Stock: 50)
```

Task 03 : Custom Report Generator

Manager: "We need different reports from the same data - a quick summary, a detailed breakdown, a low-stock alert. Build one reporting engine where the caller controls the format. Use the built-in delegates this time."

3.1 Print Reports

Write a method called `PrintReport` that accepts the product list and an Action. The method loops through all products and calls the action on each one. The caller decides what to print by passing a lambda.

Scenario 1 Short Report: Print each product as Name - \$Price

Scenario 2 Detailed Report: Print each product as [Category] Name | Price: \$X | Stock: Y

Expected Output:

```
--- Short Report ---
Laptop - $1200
Phone - $800
T-Shirt - $30
Jeans - $60
Chocolate - $5
Coffee Beans - $15

--- Detailed Report ---
[Electronics] Laptop | Price: $1200 | Stock: 10
[Electronics] Phone | Price: $800 | Stock: 25
[Clothing] T-Shirt | Price: $30 | Stock: 100
[Clothing] Jeans | Price: $60 | Stock: 50
[Food] Chocolate | Price: $5 | Stock: 200
[Food] Coffee Beans | Price: $15 | Stock: 80
[Books] C# Book | Price: $45 | Stock: 30
```

3.2. Transform Products

Write a method called `TransformProducts` that accepts the product list and a `Func`.

The method returns a `List` by applying the function to each product.

Scenario 3 Summary List: Transform each product into a string like "Laptop (\$1200)". Print all results.

Scenario 4 Price Label: Transform each product into "Expensive!" if `Price > $100`, or "Affordable" otherwise. Print each as `Name: Label`.

Expected Output:

```
--- Summary List ---  
Laptop ($1200)  
Phone ($800)  
T-Shirt ($30)  
Jeans ($60)  
Chocolate ($5)  
Coffee Beans ($15)  
C# Book ($45)  
  
--- Price Labels ---  
Laptop: Expensive!  
Phone: Expensive!  
T-Shirt: Affordable  
Novel: Affordable  
Headphones: Expensive!  
Jacket: Expensive!
```

3.3. Filter Products

Write a method called `FilterProducts` that accepts the product list and a Predicate. The method returns a List of products that match the condition.

Scenario 5 Low-Stock Alert: Find products with `Stock < 20` and print an alert for each in the format: `[LOW STOCK] Name: only X left!`

Expected Output:

```
--- Low-Stock Alert ---  
[LOW STOCK] Laptop: only 10 left!  
[LOW STOCK] Jacket: only 15 left!
```

Submission Guidelines :

- Make sure your console output matches the expected output for each task.
- Add brief comments in your code explaining which delegate type you used and why.